Secure Art Gallery — Weeks 7–10 Continuation

BIBIFI Group 11 · Huseyin Simsek and Gustavo Pepe · Fall 2026
Project: Secure Art Gallery
Stack: Python 3.12, Flask, SQLite, Jinja, Flask-Login and Flask-WTF

Starting point

The Weeks 4–6 report defined the Flask/SQLite structure, guest/employee/administrator roles, `users` distinct from `persons`, a login-only baseline, an event-sourced `GalleryEvents` design, four movement invariants and STRIDE security considerations. This implementation extends that design; the original reference document is not modified.

Week 7 — Functional Web App

Implemented: A staff-only movement form (`GET/POST /events`) that records entry and exit from the gallery and rooms. `gallery_events` stores `person_id`, nullable `room_id`, one of four event types, server-derived `recorded_by` and timestamp. `GET /dashboard` shows current room counts, `GET /occupancy` shows location counts and staff-only person names, and `GET /history` gives bounded event history. Administrators can create persons and accounts and change other users' roles. Staff can view the persons list (`GET /persons`). The history page supports person, room, and event-type filters with safe parameter binding.

State rules:

1. `gallery_enter`: person must currently be outside.
2. `room_enter`: person must be on gallery floor, the requested room must exist and have capacity.
3. `room_leave`: person must occupy that exact room.
4. `gallery_leave`: person must be on gallery floor rather than a room.

Latest accepted event determines location; no duplicated mutable occupancy field exists. State validation and insertion are performed under `BEGIN IMMEDIATE` to prevent two parallel requests from accepting the same prior state. Invalid states return **409**, do not write a movement, and create a rejected audit entry. Malformed fields return **400**. Pagination defaults to 10 rows and limits page size to **25**.

Week 8 — Security-Complete Build

Risk / control	Where implemented	Verification case
Server-side access control	`role_required` + linked guest history filtering	`test_guest_history_only_their_person`; `test_denied_403_and_admin_changes_audited`
Invalid/replayed/concurrent events	`BEGIN IMMEDIATE` + event-state logic	`test_invalid_sequence_is_409_and_unchanged`; `test_concurrent_duplicate_movements_only_one_a`
Over-capacity room	`room_enter` occupancy/capacity check	`test_capacity_enforced`
SQL injection / invalid parameters	`?` parameters and strictly parsed integer IDs	`test_request_validation_and_parameterization`
XSS	Jinja HTML escaping, CSP, no inline scripts	`test_xss_is_escaped_on_history`
CSRF	Flask-WTF on all POSTs	`test_csrf_on_state_changes`
Login brute force	database-backed 5 username failures / 20 IP failures per 15 minutes	`test_rate_limit_and_same_bad_credential_messag`
Confidentiality / sessions	`HttpOnly`, `SameSite=Lax`, 30-minute session, clear on login/logout; `Cache-Control: no-store`	`test_login_and_roles` and header tests
Repudiation	login, logout, accepted/rejected events, 403, admin changes logged with actor/target/outcome/IP	audit assertions in tests
Resource exhaustion	16 KiB POST cap; max 25 history records/page	`test_size_limit`; pagination tests
Sensitive failures	generic login errors and controlled 403/404/500 pages	route/error checks

Threat model update: Spoofing is reduced by password hashing, role-checked sessions and login rate limiting. Tampering is limited by CSRF, validated state transitions and atomic writes. Repudiation is addressed by an append-only UI audit log (not tamper-proof against DB administrators). Information disclosure is mitigated by record ownership, output escaping and error handling. DoS is reduced by request-size and history limits; local SQLite and a small worker configuration still impose availability limits. Privilege escalation is re-

duced by server-side role checks and restricted fields, including refusing admin self-demotion.

Security caveats: Classroom controls are not a professional penetration test. With the local HTTP configuration the traffic is not encrypted. Production deployment requires TLS, secure cookies, secret management, backups and monitoring. A distributed DoS, stolen session cookie or compromised database account can still cause harm; the audit log is not cryptographically protected.

Week 9 — Final Build It

Files: `README.md` (installation, roles, routes, Docker deployment), `tests/conftest.py`, `tests/functional/test_functional.py`, `tests/security/test_security.py`, GitHub Actions workflow and database migration routine in `src/backend/db.py`. The Week 6 database may be upgraded in place via `flask --app src.backend init-db` without deleting records; a backup is recommended. Local configuration and passwords remain excluded from version control.

Local run: Initialize/seed an isolated DB, log in as Employee, enter the gallery, enter/leave a room, leave the gallery, inspect occupancy and history. Log in as Guest to verify no unrelated movement is visible and direct staff/admin requests are denied. Log in as Administrator to exercise account creation/role changes and review the audit log.

Tests: This repository supplies functional and security tests. Run `python -m pytest -q` after installing `requirements-dev.txt`. GitHub Actions defines the same command for Python 3.12. Test outcomes must be recorded from an actual run; this report does not invent pass counts. The **official correctness oracle** was not in the provided source report or repo, so running the official oracle remains an external submission task. Deploy locally using the included Docker Compose configuration or the Flask development server; no public HTTPS hosting is claimed.

Week 10 — Break It: pending opponent exchange

The course requires an anonymized opponent code exchange, test-harness-based attack attempts and executable validated break cases. **No opponent code or official harness was provided**, so no opponent vulnerabilities have been confirmed. This is not a completed Break It Report. See `break_it_report_template.md` for a structured report format and `tests/breakit/` for an executable candidate example that skips until an assigned peer target is configured.

Proposed authorized investigation areas: bypassing role and record ownership checks (including user-supplied person ID), replaying duplicate/out-of-order movements, concurrent state races, CSRF on writes, input validation/SQLi, reflected/stored XSS, login rate limiting and session expiry, error information disclosure, and history pagination/oversized requests. Findings count only

after an executable test reproduces a real flaw on the assigned opponent implementation within the course's authorized scope.

Traceability

- **FR-7–FR-9:** Gallery entry/exit and room movement events — `POST /events`.
- **FR-10:** Occupancy — `/dashboard` and `/occupancy`.
- **FR-11:** Paginated/filterable event history — `/history`.
- **FR-12:** Administrator person/account/role functions — `/admin` and admin POST endpoints.
- **FR-13; SEC-10:** Valid state transitions, rejected events, atomic writes — `routes.events`.
- **SEC-11:** Record-level ownership on guest history — `routes.history`.
- **SEC-12:** Validation, injection/XSS defense, session/cookie configuration and error pages.
- **SEC-13:** Brute-force protection in `routes.login` and `login_failures` table.
- **SEC-14:** Page-size limit (`PAGE_MAX=25`).
- **SEC-15:** Audit metadata in `audit_logs` and `db.audit`.